COMPUTER ENGINEERING

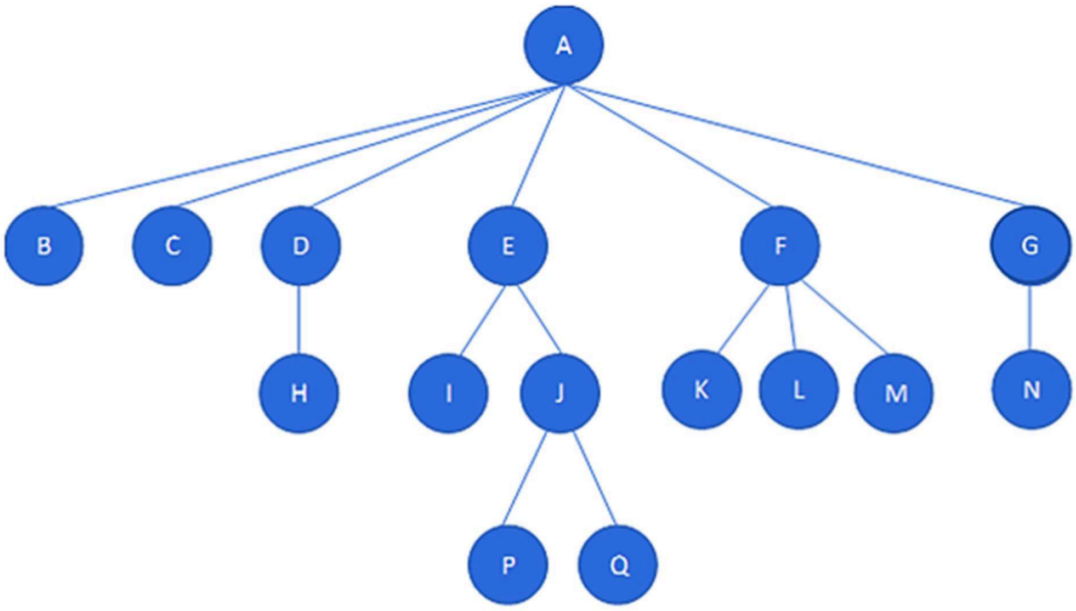
QUEZON CITY

COURSE CODE/TITLE:	CPE 010	**SECTION:**	CPE21S4
DATE PERFORMED:	9/18/2026	**DATE SUBMITTED:**	9/23/2026
NAME:	Ralph Angelov F. Braganza	**INSTRUCTOR:**	Engr. Jimlord M. Quejado

ACTIVITY NO. 9.1 TREES

1. PROCEDURE OUTPUT

ILO A: Create C++ code to implement a general tree, binary tree and binary search tree.



Task 1: Create code in C++ that will create a tree as shown in the figure above. Use linked lists as the internal representation of this tree. **Indicate your code screenshot and comments in table 9-1.**

generalTree.h

generalTree.h

procedure > generalTree.h > GeneralTree<T>

```
1  #ifndef GENERALTREE_H
2  #define GENERALTREE_H
3
4  #include <iostream>
5
6  template<typename T>
7  class GeneralTree{
8
```

```
    private:
    T key;
    //Linked list for storing the children
    struct ChildNode{
        GeneralTree<T>* child;
        ChildNode* next;
        ChildNode(GeneralTree<T>* newChild){
            child = newChild;
            next = nullptr;
        }
    };
    ChildNode* firstChild;
    //Search functions
    bool preOrderSearch(T search);
    bool inOrderSearch(T search);
    bool postOrderSearch(T search);

    public:
    //constructor
    GeneralTree(T value);
    //insertion
    void insertChild(T value);
    //get positions
    T getRootVal();
    GeneralTree<T>* getChild(int position);
    //traversal
    void preOrder();
    void postOrder();
    void inOrder();
    //search
    void findData(int choice, T search);
};

//constructor
template <typename T>
GeneralTree<T>::GeneralTree(T value){
    key = value;
    firstChild = nullptr;
}
```

```

//insert a child at the end of the linked list
template <typename T>
void GeneralTree<T>::insertChild(T value){
    GeneralTree<T>* newChild = new GeneralTree<T>(value);
    ChildNode* newNode = new ChildNode(newChild);
    //if there are no children yet
    if(firstChild == nullptr){
        firstChild = newNode;
    }
    //if there are already children
    else{
        ChildNode* current = firstChild;
        while(current->next != nullptr){
            current = current->next;
        }
        current->next = newNode;
    }
}

//get the value of the current node
template <typename T>
T GeneralTree<T>::getRootVal(){
    return key;
}

//get a child based on its position
//position starts at 0
template <typename T>
GeneralTree<T>* GeneralTree<T>::getChild(int position){
    ChildNode* current = firstChild;
    int count = 0;

    while(current != nullptr){
        if(count == position){
            return current->child;
        }
        current = current->next;
        count++;
    }
    return nullptr;
}
```

```cpp

//preorder traversal
template <typename T>
void GeneralTree<T>::preOrder(){
    //print root first
    std::cout << key << " ";
    //then visit the children from left to right
    ChildNode* current = firstChild;
    while(current != nullptr){
        current->child->preOrder();
        current = current->next;
    }
}

//postorder traversal
template <typename T>
void GeneralTree<T>::postOrder(){
    //visit the children first
    ChildNode* current = firstChild;
    while(current != nullptr){
        current->child->postOrder();
        current = current->next;
    }
    //print root last
    std::cout << key << " ";
}

//inorder traversal
//Convention: first child, root, then remaining children
template <typename T>
void GeneralTree<T>::inOrder(){
    ChildNode* current = firstChild;
    //visit the first child's subtree
    if(current != nullptr){
        current->child->inOrder();
        current = current->next;
    }
    //print the current node
    std::cout << key << " ";
    //visit the remaining children
    while(current != nullptr){
        current->child->inOrder();
        current = current->next;
    }
}
```

```cpp

//preorder search
template <typename T>
bool GeneralTree<T>::preOrderSearch(T search){
    //check the current node first
    if(key == search){
        return true;
    }
    //then search the children
    ChildNode* current = firstChild;
    while(current != nullptr){
        if(current->child->preOrderSearch(search)){
            return true;
        }
        current = current->next;
    }
    return false;
}

//inorder search
template <typename T>
bool GeneralTree<T>::inOrderSearch(T search){
    ChildNode* current = firstChild;
    //search the first child's subtree
    if(current != nullptr){
        if(current->child->inOrderSearch(search)){
            return true;
        }
        current = current->next;
    }
    //check the current node
    if(key == search){
        return true;
```

```cpp
    }
    //search the remaining children
    while(current != nullptr){
        if(current->child->inOrderSearch(search)){
            return true;
        }
        current = current->next;
    }
    return false;
}

//postorder search
template <typename T>
bool GeneralTree<T>::postOrderSearch(T search){
    //search the children first
    ChildNode* current = firstChild;
    while(current != nullptr){
        if(current->child->postOrderSearch(search)){
            return true;
        }
        current = current->next;
    }
    //check the current node last
    if(key == search){
        return true;
    }
    return false;
}
```

```cpp

//search based on the selected traversal
template <typename T>
void GeneralTree<T>::findData(int choice, T search){
    bool found = false;
    switch(choice){
        case 1:
            found = preOrderSearch(search);
            break;
        case 2:
            found = inOrderSearch(search);
            break;
        case 3:
            found = postOrderSearch(search);
            break;
        default:
            std::cout << "Invalid Choice!" << std::endl;
            return;
    }

    //display a message only if the value was found
    if(found){
        std::cout << search << " was found!" << std::endl;
    }
}

#endif
```

main.cpp.

main.cpp

procedure > main.cpp > main()

```cpp
#include <iostream>
#include "generalTree.h"

int main(){

    //Tree structure:
    //level 0
    GeneralTree<char>* root = new GeneralTree<char>('A');

    //level 1
    root->insertChild('B');
    root->insertChild('C');
    root->insertChild('D');
    root->insertChild('E');
    root->insertChild('F');
    root->insertChild('G');
```

```
    //level 2
    root->getChild(2)->insertChild('H');

    root->getChild(3)->insertChild('I');
    root->getChild(3)->insertChild('J');

    root->getChild(4)->insertChild('K');
    root->getChild(4)->insertChild('L');
    root->getChild(4)->insertChild('M');

    root->getChild(5)->insertChild('N');

    //level 3
    root->getChild(3)->getChild(1)->insertChild('P');
    root->getChild(3)->getChild(1)->insertChild('Q');

    //traverse the original tree
    std::cout << "Original Tree Traversals:" << std::endl;
    std::cout << "Preorder Order: ";
    root->preOrder();
    std::cout << "\nPostorder Order: ";
    root->postOrder();
    std::cout << "\nInorder Order: ";
    root->inOrder();

    std::cout << std::endl;

    //search the original tree
    std::cout << "\nSearch Results:" << std::endl;

    root->findData(1, 'F');
    root->findData(2, 'B');
    root->findData(3, 'Z');

    //add O as the second child of G
    root->getChild(5)->insertChild('O');

    //traverse the tree after adding O
    std::cout << "\nTraversals After Adding O:" << std::endl;
    std::cout << "Preorder Order: ";
    root->preOrder();
    std::cout << "\nPostorder Order: ";
    root->postOrder();
    std::cout << "\nInorder Order: ";
    root->inOrder();

    std::cout << std::endl;

    //search for O using the three traversal choices
    std::cout << "\nSearch Results After Adding O:" << std::endl;

    root->findData(1, 'O');
    root->findData(2, 'O');
    root->findData(3, 'O');

    return 0;
}
```

Table 9-1

Task 2: Complete the following table:

Node	Height	Depth
A	3	0
B	0	1
C	0	1
D	1	1
E	2	1
F	1	1
G	1	1
H	0	2
I	0	2
J	1	2
K	0	2
L	0	2
M	0	2
N	0	2
P	0	3
Q	0	3

Table 9-2

ILO B: Create C++ code for implementation of tree traversal methods such as pre-order, in-order and post-order traversal

Task 3: After implementing the code for above, answer the following:

3.1 Given the tree diagram, find the result of the pre-order, post-order and in-order traversal strategies by hand.Include this as table 9-3 in section 6

Pre-order	A B C D H E I J P Q F K L M G N ROOT → LEFT → RIGHT
Post-order	B C H D I P Q J E K L M F N G A LEFT → ROOT → RIGHT
In-order	B A C H D I E P J Q K F L M N G LEFT → RIGHT → ROOT

Table 9-3

3.2 Create a function for pre-order, post-order and in-order traversal. Make sure that each function displays an output into the console.

generalTree.h Functions

```
91  //preorder traversal
92  template <typename T>
93  void GeneralTree<T>::preOrder(){
94      //print root first
95      std::cout << key << " ";
96      //then visit the children from left to right
97      ChildNode* current = firstChild;
98      while(current != nullptr){
99          current->child->preOrder();
100         current = current->next;
101     }
102 }
103
104 //postorder traversal
105 template <typename T>
106 void GeneralTree<T>::postOrder(){
107     //visit the children first
108     ChildNode* current = firstChild;
109     while(current != nullptr){
110         current->child->postOrder();
111         current = current->next;
112     }
113     //print root last
114     std::cout << key << " ";
115 }
```

```cpp

//inorder traversal
//Convention: first child, root, then remaining children
template <typename T>
void GeneralTree<T>::inOrder(){
    ChildNode* current = firstChild;
    //visit the first child's subtree
    if(current != nullptr){
        current->child->inOrder();
        current = current->next;
    }
    //print the current node
    std::cout << key << " ";
    //visit the remaining children
    while(current != nullptr){
        current->child->inOrder();
        current = current->next;
    }
}
```

main.cpp

```cpp
    //traverse the original tree
    std::cout << "Original Tree Traversals:" << std::endl;
    std::cout << "Preorder Order: ";
    root->preOrder();
    std::cout << "\nPostorder Order: ";
    root->postOrder();
    std::cout << "\nInorder Order: ";
    root->inOrder();
```

Output of Functions

```
Original Tree Traversals:
Preorder Order: A B C D H E I J P Q F K L M G N
Postorder Order: B C H D I P Q J E K L M F N G A
Inorder Order: B A C H D I E P J Q K F L M N G
```

Observation (Comparison of output #1 and the output of my functions in #2)

The output of the traversal functions matches the traversal order obtained by hand in Task 3.1. Pre-order traversal starts from visiting the parent nodes before their children, whereas post-order traversal visits the children before their parents. In in-order traversal, the order is consistent with that followed in the program: the subtree of the first child, the node itself, and the rest of the children.

Table 9-4

3.3. Create a function called findData that will take the following parameter: “CHOICE, KEY”. Choice will determine which traversal method will be used to traverse the tree to find the second parameter KEY. If the KEY is found, the value will be displayed such that: “{KEY} was found!”. Otherwise, no output. Include your output as table 9-5 in section 6.

generalTree.h findData Function

```
197  //search based on the selected traversal
198  template <typename T>
199  void GeneralTree<T>::findData(int choice, T search){
200      bool found = false;
201      switch(choice){
202          case 1:
203              found = preOrderSearch(search);
204              break;
205          case 2:
206              found = inOrderSearch(search);
207              break;
208          case 3:
209              found = postOrderSearch(search);
210              break;
211          default:
212              std::cout << "Invalid Choice!" << std::endl;
213              return;
214      }
215
216      //display a message only if the value was found
217      if(found){
218          std::cout << search << " was found!" << std::endl;
219      }
220  }
221
222  #endif
```

main.cpp

```
45      //search the original tree
46      std::cout << "\nSearch Results:" << std::endl;
47      root->findData(1, 'F');
48      root->findData(2, 'B');
49      root->findData(3, 'Z'); //No output since Z is not in the tree
```

Output

```
Search Results:
F was found!
B was found!
```

Table 9-5

3.4. Add a new leaf node called which will contain the data value of “O”. Make this node the child of node G. Perform the findData function you created in #3, what can you observe about the output?

main.cpp (adding O as a child of G and then searching for O using all three traversal methods.)

```
51      //add O as the second child of G
52      root->getChild(5)->insertChild('O');
53
54      //traverse the tree after adding O
55      std::cout << "\nTraversals After Adding O:" << std::endl;
56      std::cout << "Preorder Order: ";
57      root->preOrder();
58      std::cout << "\nPostorder Order: ";
59      root->postOrder();
60      std::cout << "\nInorder Order: ";
61      root->inOrder();
62
63      std::cout << std::endl;
64
65      //search for O using the three traversal choices
66      std::cout << "\nSearch Results After Adding O:" << std::endl;
67
68      root->findData(1, 'O');  // Pre-order search
69      root->findData(2, 'O');  // In-order search
70      root->findData(3, 'O');  // Post-order search
71
72      return 0;
73  }
```

Output

```
Traversals After Adding O:
Preorder Order: A B C D H E I J P Q F K L M G N O
Postorder Order: B C H D I P Q J E K L M F N O G A
Inorder Order: B A C H D I E P J Q K F L M N G O

Search Results After Adding O:
O was found!
O was found!
O was found!
```

Observation

After adding O as a child of node G, the findData() function successfully finds it using pre-order, in-order, and post-order search. Each search displays “O was found!” because O is now part of the tree. This shows that the search function can locate a newly inserted node regardless of which traversal method is selected.

Table 9-6

2. ANSWERS TO SUPPLEMENTAL ACTIVITY

ILO C: Solve given problems using the tree data structure's implementation in C++

Step 1: Implement a binary search tree that will take the following values: 2, 3, 9, 18, 0, 1, 4, 5.

main.cpp

main.cpp

```cpp
#include <iostream>
#include "BST.h"

int main() {
    //create the root node:
    BinarySearchTree<int> tree(2);

    tree.insert(3);
    tree.insert(9);
    tree.insert(18);
    tree.insert(0);
    tree.insert(1);
    tree.insert(4);
    tree.insert(5);

    std::cout<<"preOrder: ";
    tree.preOrder();
    std::cout<<std::endl;

    std::cout<<"inOrder: ";
    tree.inOrder();
    std::cout<<std::endl;

    std::cout<<"postOrder: ";
    tree.postOrder();
    std::cout<<std::endl;

    return 0;
}
```

BST.h

BST.h

```cpp
#ifndef BST_H
#define BST_H

template <typename T>
class BinarySearchTree{
    private:
        T key;
        //pointers
        BinarySearchTree<T>* left;
        BinarySearchTree<T>* right;


    public:
        //constructor
        BinarySearchTree<T>(T value);
        void insert(T value);

        //Traversal ALgo:
        void preOrder();
        void inOrder();
        void postOrder();

};
```

```cpp

template<typename T>
    BinarySearchTree<T>::BinarySearchTree(T value){
        key = value;
        left = nullptr;
        right = nullptr;
    }

template<typename T>
void BinarySearchTree<T>::insert(T value){
    if(value < key){
        if(left == nullptr){
            left = new BinarySearchTree<T>(value);
        }
        else{
            left->insert(value);
        }
    }
    else if (value > key){
        if(right == nullptr){
            right = new BinarySearchTree<T>(value);
        }
        else{
            right->insert(value);
        }
    }
}

template<typename T>
void BinarySearchTree<T>::preOrder(){
    std::cout<<key<<" ";
    if(left !=nullptr){
        left->preOrder();
    }
    if(right !=nullptr){
        right->preOrder();
    }
}

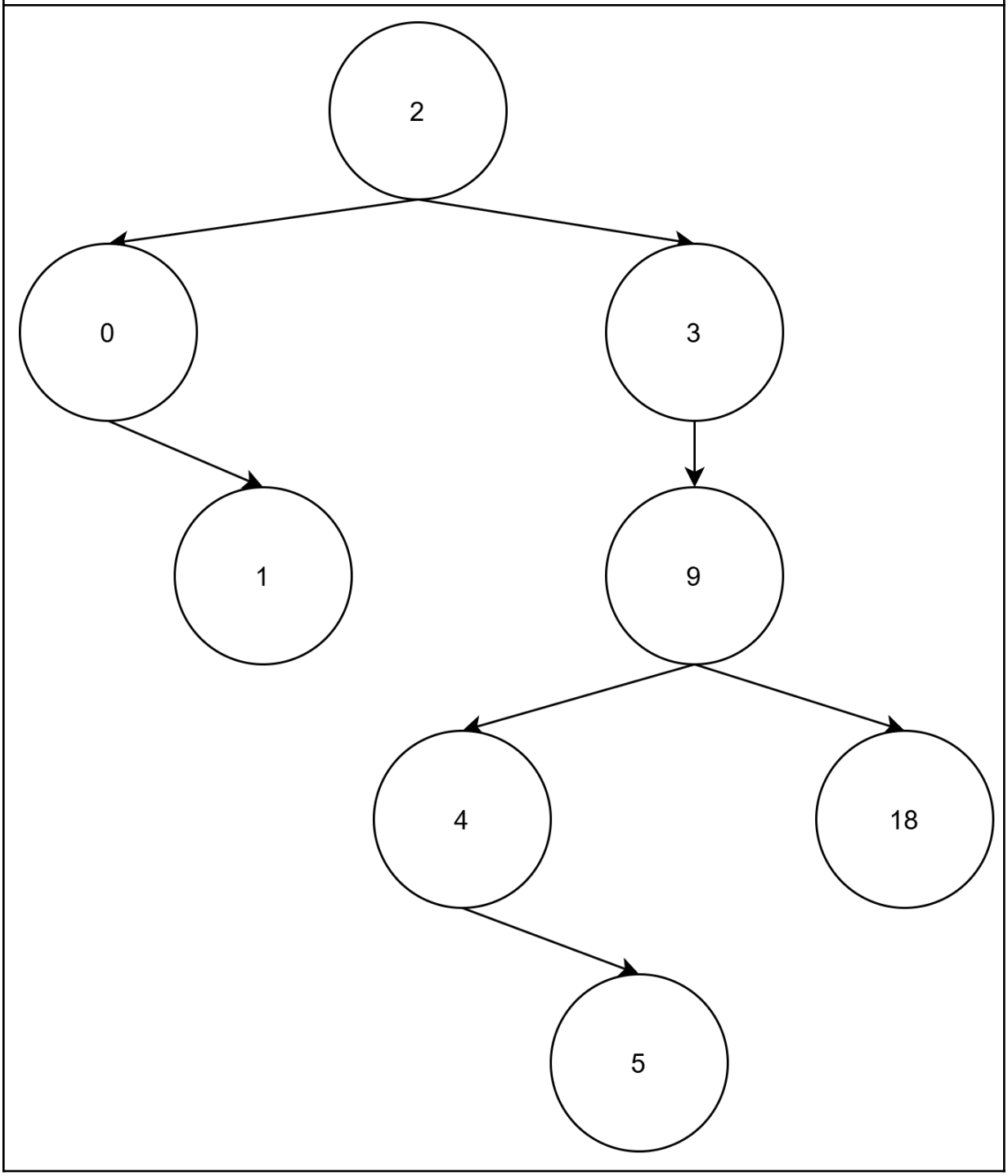
template<typename T>
void BinarySearchTree<T>::inOrder(){
    //go to left
    if(left !=nullptr){
        left->inOrder();
    }
    std::cout<<key<<" ";
    //go to right
    if(right !=nullptr){
        right->inOrder();
    }
}

template<typename T>
void BinarySearchTree<T>::postOrder(){
    //go to left
    if(left !=nullptr){
        left->postOrder();
    }

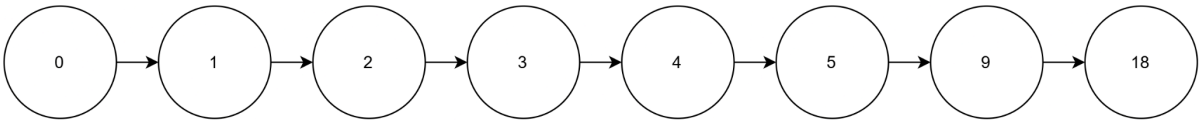
    //go to right
    if(right !=nullptr){
        right->postOrder();
    }
    std::cout<<key<<" ";
}

#endif
```

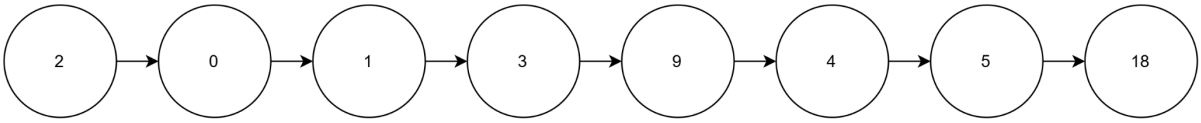
TREE DIAGRAM



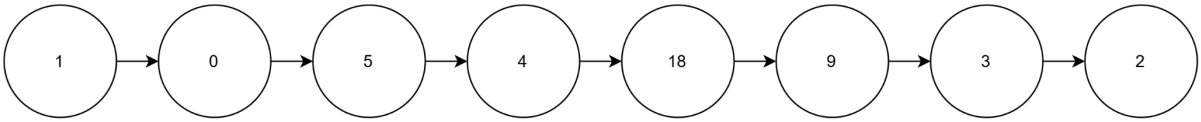
IN-ORDER (LEFT->ROOT->RIGHT)



PRE-ORDER (ROOT->LEFT->RIGHT)



POST-ORDER (LEFT->RIGHT->ROOT)



Compare the different traversal methods. In-order traversal was performed with what function?

Output

```
preOrder: 2 0 1 3 9 4 5 18
inOrder: 0 1 2 3 4 5 9 18
postOrder: 1 0 5 4 18 9 3 2
```

Pre-order Traversal

```
52  template<typename T>
53  void BinarySearchTree<T>::preOrder(){
54      std::cout<<key<<" ";
55      if(left !=nullptr){
56          left->preOrder();
57      }
58      if(right !=nullptr){
59          right->preOrder();
60      }
61  }
```

OUTPUT:

```
preOrder: 2 0 1 3 9 4 5 18
```

Post-order Traversal

```
76  template<typename T>
77  void BinarySearchTree<T>::postOrder(){
78      //go to left
79      if(left !=nullptr){
80          left->postOrder();
81      }
82
83      //go to right
84      if(right !=nullptr){
85          right->postOrder();
86      }
87      std::cout<<key<<" ";
88  }
89
```

OUTPUT:

```
postOrder: 1 0 5 4 18 9 3 2
```

In-order Traversal

```
63  template<typename T>
64  void BinarySearchTree<T>::inOrder(){
65      //go to left
66      if(left !=nullptr){
67          left->inOrder();
68      }
69      std::cout<<key<<" ";
70      //go to right
71      if(right !=nullptr){
72          right->inOrder();
73      }
74  }
```

OUTPUT:

```
inOrder: 0 1 2 3 4 5 9 18
```

Observations:

Pre-order traversal:
In pre-order traversal, the root node is visited first, then the left sub-tree and then the right sub-tree. The output is different from that of in-order and post-order traversal since it involves visiting the nodes in a different sequence.

In-order traversal:
In in-order traversal, the left sub-tree is visited first, then the root node and then the right sub-tree. The output is in ascending order since the program uses binary search tree.

Post-order traversal:
In post-order traversal, the left sub-tree and right sub-tree are visited first before visiting the root node. The output is different from that of pre-order and in-order traversal because the root is visited at last.

Comparison with the diagrams in Item 2
The pre-order and post-order outputs should be the same as the traversal sequences given in your diagram in Item 2, as long as the diagrams are based on the same BST using the given insertion order. In-order output is in ascending order.

Conclusion:

After performing this very long hands-on activity, I've obtained quite some knowledge on how to make a Binary Search Tree data structure, where values are inserted following the BST. Furthermore, I learned about the differences between pre-order, in-order, and post-order traversals. This procedure has shown me how values are being inserted into the left or right sub-trees and how recursion is applied to access all nodes. I have found out that in-order traversal prints values in the ascending order, whereas pre-order and post-order show different results based on when the root node is visited.

In my opinion, I performed this task quite well because I could build the tree and understand the results of the traversals. Nevertheless, I still need to develop my skills related to recursion and be able to draw a diagram of BST according to the given sequence of values. I also need to learn how to trace the execution process of each of the traversals. I had a lot of fun making the diagrams and learning how each order is traversed through with each random value in the tree. I'm still not that good with using the link list implementation but this activity has done wonders for me on how everything operates and gave me more knowledge about the logic behind it. Even though this activity was quite tiring it was quite fun at the same time.

3. ARTIFICIAL INTELLIGENCE (AI) USE DISCLOSURE STATEMENT

I hereby declare that artificial intelligence (AI) tools were used, if applicable, in the preparation of this academic work. The following indicates the nature and extent of AI assistance:

Tools Used: **ChatGPT, Google Overview (Gemini), QuillBot**

Examples: ChatGPT, Microsoft Copilot, Grammarly, QuillBot, etc.

Purpose of Use: **Grammar correction, idea generation, formatting, summarization, code assistance, and explanation of concepts.**

Examples: grammar correction, idea generation, formatting, summarization, code assistance, explanation of concepts, etc.

Extent of Use: **AI was used to obtain ideas on how to complete the procedure, most of the concepts were referenced from the supplementary activity and it was also used to fix grammar and misconceptions of explanation.**

Explain how AI was used and confirm that it did not replace original thinking, analysis, authorship, or completion of the required work.

By submitting this activity, I affirm that my use of AI complies with T.I.P.'s AI usage policy and does not exceed the permitted limits for similarity, automated content generation, or academic assistance.

I understand that any false, incomplete, or misleading disclosure may be subject to investigation in accordance with due process and may result in sanctions for violation of existing T.I.P. policies.

Lab activities, templates, and related instructional materials shall not be reuploaded, reproduced, distributed, or shared with external organizations or third parties without the prior express written permission of the Technological Institute of the Philippines.


T.I.P
TECHNOLOGICAL INSTITUTE OF THE PHILIPPINES
COMPUTER ENGINEERING
QUEZON CITY